

Best Programming Practice

1. Use Variables including for Fixed, User Inputs, and Results
2. Use Methods instead of writing code in the main() function
3. Proper naming conventions for all variables and methods
4. Proper Program Name and Class Name
5. Handle Checked and Unchecked Exceptions wherever possible
6. Proper Method Name which indicates action taking inputs and providing result

Sample Program 1: Create a program to find all the occurrences of a character in a string using charAt() method

- a. Take user input for the String and occurrences of the Character to find
- b. Write a method to find all the occurrences of the characters.
 - i. The logic used is to first find the number of occurrences of the character and
 - ii. then create an array to store the indexes of the character
- c. Call the method in the main and display the result

Java

```
// Program to find all the occurrences of a character in a string
import java.util.Scanner;
class StringAnalyzer {
    // Method to find all the index of a character in a string using charAt()
    // method and return them in an array
    public static int[] findAllIndexes(String text, char ch) {
        // The count is used to find the number of occurrences of the character
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            if (text.charAt(i) == ch) {
                count++;
            }
        }

        // Create an array to store the indexes of the character
        int[] indexes = new int[count];
        int j = 0;
        for (int i = 0; i < text.length(); i++) {
            if (text.charAt(i) == ch) {
                indexes[j] = i;
                j++;
            }
        }
        return indexes;
    }
}
```

```
public static void main(String[] args) {  
    // Take user input for Text and Character to check Occurrences  
    Scanner sc = new Scanner(System.in);  
    System.out.print(Enter a text: ");  
    String text = sc.nextLine();  
    System.out.print("Enter a character to find the occurrences: ");  
    char ch = sc.next().charAt(0);  
  
    // Find the occurrences of the character  
    int[] indexes = findAllIndexes(text, ch);  
  
    // Display the occurrences of the character  
    System.out.println("Indexes of the character '" + ch + "': ");  
    for (int i = 0; i < indexes.length; i++) {  
        System.out.print(indexes[i] + " ");  
    }  
}
```

Level 3 Practice Programs

1. An organization took up the exercise to find the Body Mass Index (BMI) of all the persons in a team of 10 members. For this create a program to find the BMI and display the height, weight, BMI, and status of each individual

Hint =>

- a. Take user input in double for the weight (in kg) of the person and height (in cm) for the person and store it in the corresponding 2D array of 10 rows. The First Column stores the weight and the second column stores the height in cm
- b. Create a Method to find the BMI and status of every person given the person height and weight and return the 2D String array. Use the formula $BMI = \text{weight} / (\text{height} * \text{height})$. Note unit is kg/m^2 . For this convert cm to meter
- c. Create a Method that takes the 2D array of height and weight as parameter. Calls the user defined method to compute the BMI and the BMI Status and store in 2D String array of height, weight, BMI, and status.
- d. Create a method to display the 2D string array in a tabular format of Person Height, Weight, BMI and the Status
- e. Finally, the main function takes user inputs, calls the user-defined methods, and displays the result.

BMI	Status
≤ 18.4	Underweight
18.5 - 24.9	Normal
25.0 - 39.9	Overweight
≥ 40.0	Obese

```
import java.util.Scanner;

public class BMICalculator4 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // creating Array detail to store height (1st column), weight (2nd
        column), BMI (3rd column)
        double[][] details = new double[10][3];

        //creating Array to store status
        String[] statuses = new String[10];
```

```
// Taking input for height and weight
for (int i = 0; i < 10; i++) {
    System.out.println("Person " + (i + 1) + ":");
    System.out.print("Enter height (meters): ");
    details[i][0] = input.nextDouble();
    System.out.print("Enter weight (kg): ");
    details[i][1] = input.nextDouble();

    // Validating inputs
    if (details[i][0] <= 0 || details[i][1] <= 0) {
        System.out.println("Invalid input. Try again.");
        i--; // Retrying for the current person
        continue;
    }
}

//iterating through the details to calculate BMI's and
assigning statuses
for(int i=0; i<10; i++) {

    //calling the method calculateBMI to calculate BMI
    details[i][2] = calculateBMI(details[i][0],
details[i][1]);

    //calling the method assignStatus to assign status
    statuses[i] = assignStatus(details[i][2]);
}

// Displaying results
System.out.println("\nResults:");
for (int i = 0; i < 10; i++) {
    System.out.println("Person " + (i + 1) + ":");
    System.out.println("Height: " + details[i][0] + " meters");
    System.out.println("Weight: " + details[i][1] + " kg");
    System.out.println("BMI: " + details[i][2]);
    System.out.println("Status: " + statuses[i]);
}
```

```

        //closing the input object
        input.close();
    }

    //method to calculate BMI
    public static double calculateBMI(double height, double weight) {

        //variable to indicate BMI
        double BMI;

        // Calculating BMI
        BMI = weight / (height * height);

        //returning the BMI
        return BMI;
    }

    //method to assign weight status
    public static String assignStatus(double BMI) {

        //variable to indicate status
        String status;

        //assigning weight status
        if (BMI <= 18.4) {
            status = "Underweight";
        } else if (BMI <= 24.9) {
            status = "Normal";
        } else if (BMI <= 39.9) {
            status = "Overweight";
        } else {
            status = "Obese";
        }

        //returning the status
        return status;
    }
}

```

2. Find unique characters in a string using charAt() method and display the result

Hint =>

- a. Create a Method to find the length of the text without using the String method length()
- b. Create a method to Find unique characters in a string using charAt() method and return them as 1D array. The logic used here is as follows:
 - i. Create an array to store the unique characters in the text. The size is the length of the text
 - ii. Loops to Find the unique characters in the text. Find the unique characters in the text using a nested loop. An outer loop iterates through each character and an inner loop checks if the character is unique by comparing it with the previous characters. If the character is unique, it is stored in the result array
 - iii. Create a new array to store the unique characters
- c. Finally, the main function takes user inputs, calls the user-defined methods, and displays the result.

```
import java.util.Scanner;

public class UniqueCharacters {
    // Main method
    public static void main(String[] args) {
        // Creating Scanner object
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String text = input.nextLine();

        // Finding unique characters
        char[] uniqueChars = findUniqueCharacters(text);

        // Displaying the result
        System.out.print("Unique characters: ");
        for (char ch : uniqueChars) {
            System.out.print(ch + " ");
        }

        // Closing the Scanner
        input.close();
    }

    // Method to find the length of the string without using the built-in
    length() method
    public static int findLength(String text) {
```

```

    int length = 0;
    try {
        while (true) {
            text.charAt(length);
            length++;
        }
    } catch (IndexOutOfBoundsException e) {
        // Reached the end of the string
    }
    return length;
}

// Method to find unique characters in a string using charAt()
public static char[] findUniqueCharacters(String text) {
    int length = findLength(text);
    char[] uniqueChars = new char[length];
    int uniqueCount = 0;

    for (int i = 0; i < length; i++) {
        char currentChar = text.charAt(i);
        boolean isUnique = true;

        // Checking if the character has already been added to the
        unique array
        for (int j = 0; j < uniqueCount; j++) {
            if (uniqueChars[j] == currentChar) {
                isUnique = false;
                break;
            }
        }

        // If the character is unique, adding it to the uniqueChars
        array
        if (isUnique) {
            uniqueChars[uniqueCount] = currentChar;
            uniqueCount++;
        }
    }

    // Creating a new array to store only the unique characters
    char[] result = new char[uniqueCount];

```

```

        for (int i = 0; i < uniqueCount; i++) {
            result[i] = uniqueChars[i];
        }
        return result;
    }
}

```

3. Write a program to find the first non-repeating character in a string and show the result

Hint =>

- a. Non-repeating character is a character that occurs only once in the string
- b. Create a Method to find the first non-repeating character in a string using charAt() method and return the character. The logic used here is as follows:
 - i. Create an array to store the frequency of characters in the text. ASCII values of characters are used as indexes in the array to store the frequency of each character. There are 256 ASCII characters
 - ii. Loop through the text to find the frequency of characters in the text
 - iii. Loop through the text to find the first non-repeating character in the text by checking the frequency of each character
- c. In the main function take user inputs, call user-defined methods, and displays result.

```

import java.util.Scanner;

public class FirstUniqueCharacter {
    // Main method
    public static void main(String[] args) {
        // Creating a Scanner object for user input
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String text = input.nextLine();

        // Finding the first non-repeating character
        char result = findFirstNonRepeatingCharacter(text);

        // Displaying the result
        if (result != '\0') {
            System.out.println("The first non-repeating character is: " +
result);
        }
    }
}

```



```

    } else {
        System.out.println("No non-repeating character found.");
    }

    // Closing the Scanner
    input.close();
}

// Method to find the first non-repeating character in a string
public static char findFirstNonRepeatingCharacter(String text) {
    // creating Array to store frequency of each character (ASCII
values 0-255)
    int[] frequency = new int[256];

    // Calculating the frequency of each character in the text
    for (int i = 0; i < text.length(); i++) {
        char currentChar = text.charAt(i);
        frequency[currentChar]++;
    }

    // Finding the first character with a frequency of 1
    for (int i = 0; i < text.length(); i++) {
        char currentChar = text.charAt(i);
        if (frequency[currentChar] == 1) {
            return currentChar; // Return the first non-repeating
character
        }
    }

    // Returning null character if no non-repeating character is
found
    return '\0';
}
}

```

4. Write a program to find the frequency of characters in a string using charAt() method and display the result

Hint =>

- a. Create a method to find the frequency of characters in a string using charAt() method and return the characters and their frequencies in a 2D array. The logic used here is as follows:

- i. Create an array to store the frequency of characters in the text. ASCII values of characters are used as indexes in the array to store the frequency of each character. There are 256 ASCII characters
 - ii. Loop through the text to find the frequency of characters in the text
 - iii. Create an array to store the characters and their frequencies
 - iv. Loop through the characters in the text and store the characters and their frequencies
 - b. In the main function take user inputs, call user-defined methods, and displays result.
5. Write a program to find the frequency of characters in a string using unique characters and display the result

Hint =>

- a. Create a method to Find unique characters in a string using charAt() method and return them as 1D array. Use Nested Loops to find the unique characters in the text
- b. Create a method to find the frequency of characters in a string and return the characters and their frequencies in a 2D array. The logic used here is as follows:
 - i. Create an array to store the frequency of characters in the text. ASCII values of characters are used as indexes in the array to store the frequency of each character. There are 256 ASCII characters
 - ii. Loop through the text to find the frequency of characters in the text
 - iii. Call the uniqueCharacters() method to find the unique characters in the text
 - iv. Create a 2D String array to store the unique characters and their frequencies.
 - v. Loop through the unique characters and store the characters and their frequencies
- c. In the main function take user inputs, call user-defined methods, and displays result.

```
import java.util.Scanner;

public class CharacterFrequencyWithUnique {

    // Main method
    public static void main(String[] args) {
        // Creating a Scanner object for user input
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String text = input.nextLine();

        // Finding character frequencies
        String[][] frequencies = findCharacterFrequencies(text);

        // Displaying the result
```

```

System.out.println("Character Frequencies:");
for (String[] freq : frequencies) {
    System.out.println(freq[0] + " -> " + freq[1]);
}

// Closing the Scanner
input.close();
}

// Method to find unique characters in a string using charAt()
public static char[] uniqueCharacters(String text) {
    int length = text.length();
    char[] tempUnique = new char[length];
    int uniqueCount = 0;

    for (int i = 0; i < length; i++) {
        char currentChar = text.charAt(i);
        boolean isUnique = true;

        // Checking if the character has already been encountered
        for (int j = 0; j < uniqueCount; j++) {
            if (tempUnique[j] == currentChar) {
                isUnique = false;
                break;
            }
        }

        // If the character is unique, adding it to the tempUnique
        array
        if (isUnique) {
            tempUnique[uniqueCount] = currentChar;
            uniqueCount++;
        }
    }

    // Creating an array of the exact size to hold the unique
    characters
    char[] uniqueChars = new char[uniqueCount];
    System.arraycopy(tempUnique, 0, uniqueChars, 0, uniqueCount);
    return uniqueChars;
}

```

```
// Method to find the frequency of characters in a string
public static String[][] findCharacterFrequencies(String text) {
    // Finding unique characters
    char[] uniqueChars = uniqueCharacters(text);

    // Creating a 2D array to store unique characters and their
    // frequencies
    String[][] result = new String[uniqueChars.length][2];

    for (int i = 0; i < uniqueChars.length; i++) {
        char uniqueChar = uniqueChars[i];
        int frequency = 0;

        // Counting frequency of the unique character in the text
        for (int j = 0; j < text.length(); j++) {
            if (text.charAt(j) == uniqueChar) {
                frequency++;
            }
        }

        // Storing the character and its frequency
        result[i][0] = String.valueOf(uniqueChar);
        result[i][1] = String.valueOf(frequency);
    }
    return result;
}
```

6. Write a program to find the frequency of characters in a string using nested loops and display the result

Hint =>

- a. Create a method to find the frequency of characters in a string and return the characters and their frequencies in a 1D array. The logic used here is as follows:
 - i. Create an array to store the frequency of each character in the text and an array to store the characters in the text using the `toCharArray()` method
 - ii. Loops to Find the frequency of each character in the text and store the result in an frequency array. For this use Nested Loop with Outer loop to iterate through each character in the text and initialize the frequency of each character to 1. And an Inner loop to check for duplicate characters. In case of duplicate increment the frequency value and set the duplicate characters to '0' to avoid counting them again.

- iii. Create an 1D String array to store the characters and their frequencies. For this Iterate through the characters in the text and store the characters and their frequencies
- b. Finally, the main function takes user inputs, calls the user-defined methods, and displays the result.

```
import java.util.Scanner;

public class CharacterFrequencyNested {

    // Main method
    public static void main(String[] args) {
        // Creating a Scanner object for user input
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String text = input.nextLine();

        // Finding character frequencies
        String[] frequencies = findCharacterFrequencies(text);

        // Displaying the result
        System.out.println("Character Frequencies:");
        for (String freq : frequencies) {
            System.out.println(freq);
        }

        // Closing the Scanner
        input.close();
    }

    // Method to find the frequency of characters in a string
    public static String[] findCharacterFrequencies(String text) {
        char[] characters = text.toCharArray(); // Convert string to char
        array
        int[] frequencies = new int[characters.length]; // Array to store
        frequencies

        // Outer loop to iterate through each character
        for (int i = 0; i < characters.length; i++) {
            if (characters[i] == '0') { // Skip already processed
            characters
```

```

        continue;
    }

    // Initializing frequency to 1 for the current character
    frequencies[i] = 1;

    // Inner loop to check for duplicate characters
    for (int j = i + 1; j < characters.length; j++) {
        if (characters[i] == characters[j]) {
            frequencies[i]++;
            characters[j] = '0'; // Mark duplicate as processed
        }
    }
}

// Creating a 1D String array to store characters and their
frequencies
int uniqueCount = 0;
for (char c : characters) {
    if (c != '0') uniqueCount++;
}

String[] result = new String[uniqueCount];
int index = 0;

// Storing unique characters and their frequencies in the result
array
for (int i = 0; i < characters.length; i++) {
    if (characters[i] != '0') {
        result[index] = characters[i] + " -> " + frequencies[i];
        index++;
    }
}

return result;
}
}

```

7. Write a program to to check if a text is palindrome and display the result

Hint =>

- a. Palindrome is a word, phrase, number, or other sequence of characters that reads the same forward and backward
- b. **Logic 1:** Write a method to compare the characters from the start and end of the string to determine the text is palindrome. The logic used here is as follows:
 - i. Set the start and end indexes of the text
 - ii. Loop through the text and compare the characters from the start and the end of the string. If the characters are not equal, return false

```
import java.util.Scanner;

public class PalindromeLogic1 {

    // Main method
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String text = input.nextLine();

        // Checking palindrome using Logic 1
        if (isPalindrome(text)) {
            System.out.println(text + " is a palindrome.");
        } else {
            System.out.println(text + " is not a palindrome.");
        }

        //closing the input
        input.close();
    }

    // Method to check if a string is palindrome by comparing start and end
    // characters
    public static boolean isPalindrome(String text) {
        int start = 0;
        int end = text.length() - 1;

        while (start < end) {
            if (text.charAt(start) != text.charAt(end)) {
                return false;
            }
            start++;
            end--;
        }
    }
}
```

```
        return true;
    }
}
```

- c. **Logic 2:** Write a recursive method to compare the characters from the start and end of the text passed as parameters using recursion. The logic used here is as follows:
- First, check if the start index is greater than or equal to the end index, then return true.
 - If the characters at the start and end indexes are not equal, return false.
 - Otherwise, call the method recursively with the start index incremented by 1 and the end index

```
import java.util.Scanner;

public class PalindromeLogic2 {
    // Main method
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String text = input.nextLine();

        // Checking palindrome using Logic 2 (Recursive)
        if (isPalindrome(text, 0, text.length() - 1)) {
            System.out.println(text + " is a palindrome.");
        } else {
            System.out.println(text + " is not a palindrome.");
        }

        input.close();
    }
    // Recursive method to check if a string is palindrome
    public static boolean isPalindrome(String text, int start, int end) {
        if (start >= end) {
            return true;
        }
        if (text.charAt(start) != text.charAt(end)) {
            return false;
        }
    }
}
```



```
        return isPalindrome(text, start + 1, end - 1);
    }
}
```

- d. **Logic 3:** Write a Method to compare the characters from the start and end of the text using character arrays. The logic used here is as follows:
 - i. Firstly Write a Method to reverse a string using charAt() method and return the reversal array.
 - ii. Create a character array using String method toCharArray() and also create a reverse array. Compare the characters in the original and reverse arrays to do Palindrome check
- e. Finally in the main method do palindrom check using the three logic and display result

```
import java.util.Scanner;

public class PalindromeLogic3 {
    // Main method
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String text = input.nextLine();

        // Checking palindrome using Logic 3 (Character Arrays)
        if (isPalindrome(text)) {
            System.out.println(text + " is a palindrome.");
        } else {
            System.out.println(text + " is not a palindrome.");
        }

        input.close();
    }

    // Method to reverse a string and return the character array
    public static char[] reverseString(String text) {
        char[] reversed = new char[text.length()];
        int reverseIndex = 0;

        for (int i = text.length() - 1; i >= 0; i--) {
            reversed[reverseIndex++] = text.charAt(i);
        }
    }
}
```

```

    }
    return reversed;
}

// Method to check if a string is palindrome by comparing original and
reversed arrays
public static boolean isPalindrome(String text) {
    char[] original = text.toCharArray();
    char[] reversed = reverseString(text);

    for (int i = 0; i < original.length; i++) {
        if (original[i] != reversed[i]) {
            return false;
        }
    }
    return true;
}
}

```

8. Write a program to check if two texts are anagrams and display the result

Hint =>

- a. Anagram is a word or phrase formed by rearranging the same letters to form different words or phrase,
- b. Write a method to check if two texts are anagrams. The logic used here is as follows:
 - i. Check if the lengths of the two texts are equal
 - ii. Create an array to store the frequency of characters in the strings for the two text
 - iii. Find the frequency of characters in the two texts using the loop
 - iv. Compare the frequency of characters in the two texts. If the frequencies are not equal, return false
- c. In the main function take user inputs, call user-defined methods, and displays result.

```

import java.util.Scanner;

public class AnagramCheck {
    // Main method
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Taking user inputs for two texts
    }
}

```

```

System.out.print("Enter the first text: ");
String text1 = input.nextLine();

System.out.print("Enter the second text: ");
String text2 = input.nextLine();

// Checking if the two texts are anagrams
if (areAnagrams(text1, text2)) {
    System.out.println(text1 + " and " + text2 + " are anagrams.");
} else {
    System.out.println(text1 + " and " + text2 + " are not
anagrams.");
}

input.close();
}
// Method to check if two texts are anagrams
public static boolean areAnagrams(String text1, String text2) {
    // Checking if the lengths of the two texts are equal
    if (text1.length() != text2.length()) {
        return false;
    }

    // Creating arrays to store the frequency of characters
    int[] freqText1 = new int[256];
    int[] freqText2 = new int[256];

    // Finding the frequency of characters in the two texts
    for (int i = 0; i < text1.length(); i++) {
        freqText1[text1.charAt(i)]++;
        freqText2[text2.charAt(i)]++;
    }

    // Comparing the frequency of characters in both texts
    for (int i = 0; i < 256; i++) {
        if (freqText1[i] != freqText2[i]) {
            return false;
        }
    }

    // If all frequencies match, the texts are anagrams

```

```
        return true;
    }
}
```

9. Create a program to display a calendar for a given month and year. The program should take the month and year as input from the user and display the calendar for that month. E.g. for 07 2005 user input, the program should display the calendar as shown below

```
July 2005
Sun Mon Tue Wed Thu Fri Sat
    1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
```

Hint =>

- Write a Method to get the name of the month. For this define a month Array to store the names of the months
- Write a Method to get the number of days in the month. For this define a days Array to store the number of days in each month. For Feb month, check for Leap Year to get the number of days. Also, define a Leap Year Method.
- Write a method to get the first day of the month using the Gregorian calendar algorithm

$$y0 = y - (14 - m) / 12$$

$$x = y0 + y0/4 - y0/100 + y0/400$$

$$m0 = m + 12 \times ((14 - m) / 12) - 2$$

$$d0 = (d + x + 31m0 / 12) \bmod 7$$

- Displaying the Calendar requires 2 **for** loops.
 - The first **for** loop up to the first day to get the proper indentation. As in the example above 3 spaces from Sun to Thu as to be set as July 1st starts on Fri
 - The Second **for** loop Displays the days of the month starting from 1 to the number of days. Add proper indentation for single-digit days using **%3d** to display the integer right-justified in a field of width 3. Please note to move to the next line after Sat

```
import java.util.Scanner;

public class CalendarProgram {
```

```
// Main method
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

    // Taking user input for month and year
    System.out.print("Enter the month (1-12): ");
    int month = input.nextInt();
    System.out.print("Enter the year: ");
    int year = input.nextInt();

    // Displaying the calendar for the given month and year
    displayCalendar(month, year);

    input.close();
}

// Method to get the name of the month
public static String getMonthName(int month) {
    String[] months = {
        "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November",
        "December"
    };
    return months[month - 1];
}

// Method to check if the given year is a leap year
public static boolean isLeapYear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// Method to get the number of days in a month considering leap year
// for February
public static int getNumberOfDaysInMonth(int month, int year) {
    int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    // If the month is February, checking for a leap year
    if (month == 2 && isLeapYear(year)) {
        return 29; // Leap year, February has 29 days
    }

    return days[month - 1];
}
```

```

    }

    // Method to get the first day of the month using the Gregorian
    calendar algorithm
    public static int getFirstDayOfMonth(int month, int year) {
        // Using the Gregorian calendar formula to find the first day of
        the month
        if (month < 3) {
            month += 12;
            year--;
        }

        int y0 = year - (14 - month) / 12;
        int x = y0 + y0 / 4 - y0 / 100 + y0 / 400;
        int m0 = month + 12 * ((14 - month) / 12) - 2;
        int d0 = (1 + x + 31 * m0 / 12) % 7;

        // Returning the day of the week (0 = Sunday, 1 = Monday, ...,
        6 = Saturday)
        return d0;
    }

    // Method to display the calendar for the given month and year
    public static void displayCalendar(int month, int year) {
        // Getting the first day of the month and number of days in the
        month
        int firstDay = getFirstDayOfMonth(month, year);
        int numDays = getNumberOfDaysInMonth(month, year);
        String monthName = getMonthName(month);

        // Displaying the month and year
        System.out.println("Calendar for " + monthName + " " + year);
        System.out.println("Sun Mon Tue Wed Thu Fri Sat");

        // Printing leading spaces for the first day of the month
        int dayOfWeek = firstDay;
        for (int i = 0; i < dayOfWeek; i++) {
            System.out.print("    ");
        }

        // Printing the days of the month
    }

```

```

    for (int day = 1; day <= numDays; day++) {
        System.out.printf("%3d ", day);
        dayOfWeek++;
        if (dayOfWeek == 7) {
            // Moving to the next line after Saturday
            System.out.println();
            dayOfWeek = 0;
        }
    }

    // Moving to the next line after printing all days
    System.out.println();
}
}

```

10. Write a program to create a deck of cards, initialize the deck, shuffle the deck, and distribute the deck of n cards to x number of players. Finally, print the cards the players have.

Hint =>

- Create a deck of cards with suits "Hearts", "Diamonds", "Clubs", "Spades" and ranks from "2", "3", "4", "5", "6", "7", "8", "9", "10", "Jack", "Queen", "King", and "Ace"
- Calculate the number of cards in the deck and initialize the deck

```
int numOfCards = suits.length * ranks.length;
```

- Write a Method to Initialize the deck of cards with suits and ranks and return the deck. The deck is an array of strings where each string represents a card in the deck represented as "rank of suit" e.g., "2 of Hearts"
- Write a Method to Shuffle the deck of cards and return the shuffled deck. To shuffle the card iterate over the deck and swap each card with a random card from the remaining deck to shuffle the deck. Please find the steps below

Step1: Use for Loop Iterate over the deck and swap each card with a random card from the remaining deck

Step 2: Inside the Loop Generate a random card number between i and n using the following code

```
int randomCardNumber = i + (int) (Math.random() * (n - i));
```

Step 3: Swap the current card with the random card

- Write a Method to distribute the deck of n cards to x number of players and return the players. For this Check the n cards can be distributed to x players. If possible then Create a 2D array to store the players and their cards

f. Write a Method to Print the players and their cards

```
import java.util.Random;

public class CardDeck {

    // Method to initialize the deck of cards
    public static String[] initializeDeck() {
        String[] suits = {"Hearts", "Diamonds", "Clubs", "Spades"};
        String[] ranks = {"2", "3", "4", "5", "6", "7", "8", "9", "10",
"Jack", "Queen", "King", "Ace"};

        String[] deck = new String[suits.length * ranks.length];
        int index = 0;

        // Initializing the deck by combining ranks and suits
        for (String suit : suits) {
            for (String rank : ranks) {
                deck[index++] = rank + " of " + suit;
            }
        }
        return deck;
    }

    // Method to shuffle the deck of cards
    public static void shuffleDeck(String[] deck) {
        Random rand = new Random();

        // Shuffling the deck by swapping each card with a random card from
the remaining deck
        for (int i = 0; i < deck.length; i++) {
            // Generating a random index between i and deck.length
            int randomCardIndex = i + rand.nextInt(deck.length - i);
            String temp = deck[i];
            deck[i] = deck[randomCardIndex];
            deck[randomCardIndex] = temp;
        }
    }

    // Method to distribute cards to players
    public static String[][] distributeCards(String[] deck, int n, int x) {
        // Checking if the number of cards can be evenly distributed to the
```



```

players
    if (n % x != 0) {
        System.out.println("Cards cannot be evenly distributed.");
        return new String[0][0]; // Return an empty array if
distribution is not possible
    }

    // Creating a 2D array to store the players' hands
    String[][] players = new String[x][n / x];

    // Distributing the cards to players
    int cardIndex = 0;
    for (int i = 0; i < x; i++) {
        for (int j = 0; j < n / x; j++) {
            players[i][j] = deck[cardIndex++];
        }
    }

    return players;
}

// Method to print the players' cards
public static void printPlayersCards(String[][] players) {
    int playerNumber = 1;
    for (String[] player : players) {
        System.out.println("Player " + playerNumber + " has: ");
        for (String card : player) {
            System.out.println(card);
        }
        System.out.println();
        playerNumber++;
    }
}

public static void main(String[] args) {
    // Total number of cards in the deck
    int n = 52;
    // Number of players
    int x = 4;

    // Initializing, shuffling, and distributing cards

```

```
String[] deck = initializeDeck();
shuffleDeck(deck);
String[][] players = distributeCards(deck, n, x);

// Printing the cards each player has
if (players.length > 0) {
    printPlayersCards(players);
}
}
```